

Lecture 2: Elements of Deep Learning
Networks Learn the Data Distribution
Hello World: MNIST

Deep Learning in Classical and Quantum Physics

Marcin Płodzień

Atomic Optics Department
Institute of Theoretical Physics
Jagiellonian University in Kraków

1

Elements of Deep Learning



JAGIELLONIAN UNIVERSITY
IN KRAKOW

Elements of Deep Learning

Starting point: dataset

$$\mathcal{D} = \cup_{i=1}^{N_b} \mathcal{D}^{(j)}, \quad \mathcal{D}^{(j)} = \left\{ (\vec{x}_i^{(j)}, \vec{y}_i^{(j)}) \right\}, \; i = 1 \dots N_{\mathcal{D}^{(j)}}$$

Problem definition: **regression or classification**?

1. Loss function: what do we minimize? $\mathcal{L}^{(j)}(\vec{\theta}_t) = \frac{1}{N_{\mathcal{D}^{(j)}}} \sum_{i=1}^{N_{\mathcal{D}^{(j)}}} \mathcal{L}_i \left(\vec{y}_i^{(j)}, \bar{\vec{y}}_i^{(j)} \right)$
2. Neural network architecture
3. Training algorithm

Elements of Deep Learning

Starting point: dataset

$$\mathcal{D} = \cup_{i=1}^{N_b} \mathcal{D}^{(j)}, \quad \mathcal{D}^{(j)} = \left\{ (\vec{x}_i^{(j)}, \vec{y}_i^{(j)}) \right\}, \; i = 1 \dots N_{\mathcal{D}^{(j)}}$$

Problem definition: regression or classification?

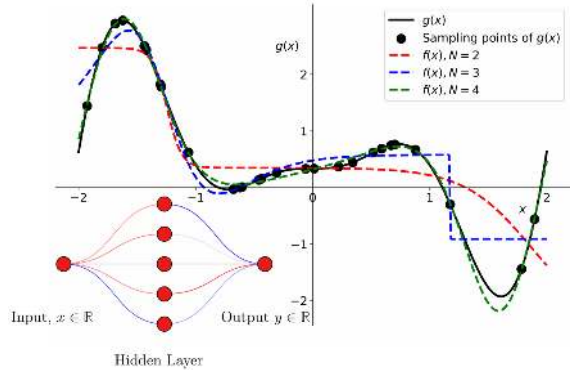
1. **Loss function**: what do we minimize? $\mathcal{L}^{(j)}(\vec{\theta}_t) = \frac{1}{N_{\mathcal{D}^{(j)}}} \sum_{i=1}^{N_{\mathcal{D}^{(j)}}} \mathcal{L}_i\left(\vec{y}_i^{(j)}, \bar{\vec{y}}_i^{(j)}\right)$
2. Neural network architecture
3. Training algorithm

Loss Function: Regression

Non-linear curve fitting problem

$$f_{\vec{\theta}}(\vec{x}) = \bar{\vec{y}} \approx \vec{y}.$$

$$\mathcal{L}_{\text{MSE}} = \frac{1}{N}\sum_{i=1}^{N}\left(\vec{y}_i - \bar{\vec{y}}_i\right)^2$$



Loss Function: Classification



Class 0



Class 1

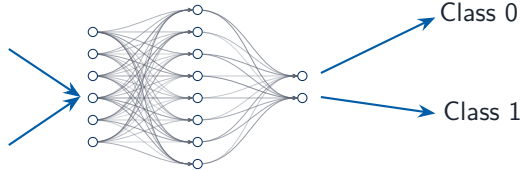
Loss Function: Classification





× Class 0

× Class 1



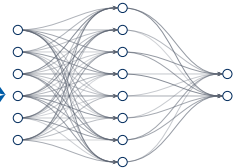
Loss Function: Classification





x $p(x) = [1,0]$

x $p(x) = [0,1]$



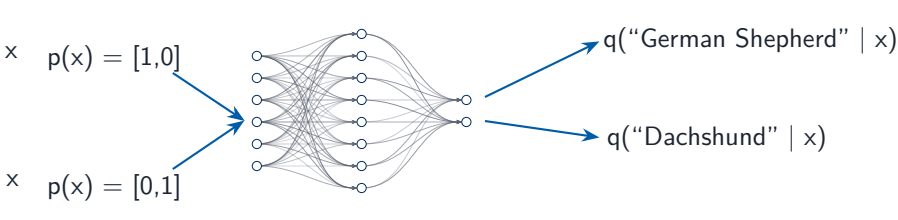
q(“German Shepherd” | x)

q(“Dachshund” | x)

Loss Function: Classification







$p(x)$: Probability distribution of training data

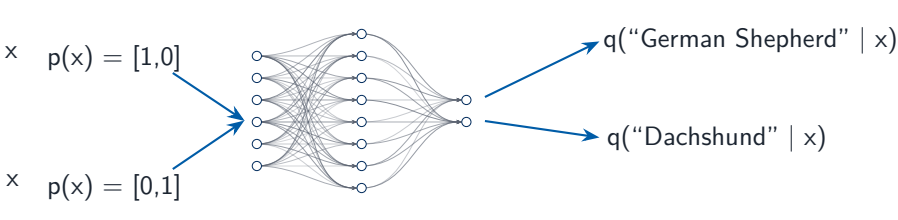
$q(x)$: Probability distribution given by NN

Kullback – Leibler divergence: $D_{\mathrm{KL}}(p\|q) = \left\langle \log \frac{p(x)}{q(x)} \right\rangle_p = \frac{1}{n} \sum_i p(x_i) \log \frac{p(x_i)}{q(x_i)}$

Loss Function: Classification







$p(x)$: Probability distribution of training data

$q(x)$: Probability distribution given by NN

Kullback – Leibler divergence: $D_{\mathrm{KL}}(p\|q) = \left\langle \log \frac{p(x)}{q(x)} \right\rangle_p = \frac{1}{n} \sum_i p(x_i) \log \frac{p(x_i)}{q(x_i)}$

1. Vanishes for identical $q(x) = p(x)$
2. Asymmetric $D_{\mathrm{KL}}(p\|q) \neq D_{\mathrm{KL}}(q\|p)$

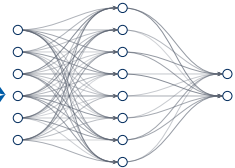
Loss Function: Classification





× p(x) = [1,0]

× p(x) = [0,1]



q("German Shepherd" | x)

q("Dachshund" | x)

$p(x)$: Probability distribution of training data

$q(x)$: Probability distribution given by NN

Kullback – Leibler divergence: $D_{\mathrm{KL}}(p\|q) = \left\langle \log \frac{p(x)}{q(x)} \right\rangle_p = \frac{1}{n} \sum_i p(x_i) \log \frac{p(x_i)}{q(x_i)}$

1. Vanishes for identical $q(x) = p(x)$

2. Asymmetric $D_{\mathrm{KL}}(p\|q) \neq D_{\mathrm{KL}}(q\|p)$

Categorical cross-entropy

$$D_{\mathrm{KL}}(p\|q) = \frac{1}{n} \sum_i p(x_i) \log p(x_i) - \frac{1}{n} \sum_i p(x_i) \log q(x_i) =: \cancel{-S(p)} + \underbrace{\mathcal{L}_{\mathrm{CE}}(p,q)}$$

Loss function of interest

Elements of Deep Learning

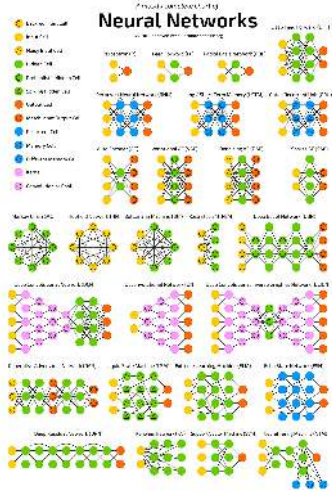
Starting point: dataset

$$\mathcal{D} = \cup_{i=1}^{N_b} \mathcal{D}^{(j)}, \quad \mathcal{D}^{(j)} = \left\{ (\vec{x}_i^{(j)}, \vec{y}_i^{(j)}) \right\}, \; i = 1 \dots N_{\mathcal{D}^{(j)}}$$

Problem definition: regression or classification?

1. Loss function: what do we minimize? $\mathcal{L}^{(j)}(\vec{\theta}_t) = \frac{1}{N_{\mathcal{D}^{(j)}}} \sum_{i=1}^{N_{\mathcal{D}^{(j)}}} \mathcal{L}_i\left(\vec{y}_i^{(j)}, \bar{\vec{y}}_i^{(j)}\right)$
2. **Neural network architecture**
3. Training algorithm

Zoology of NN Architectures



Elements of Deep Learning

Starting point: dataset

$$\mathcal{D} = \cup_{i=1}^{N_b} \mathcal{D}^{(j)}, \quad \mathcal{D}^{(j)} = \left\{ (\vec{x}_i^{(j)}, \vec{y}_i^{(j)}) \right\}, \ i = 1 \dots N_{\mathcal{D}^{(j)}}$$

Problem definition: regression or classification?

1. Loss function: what do we minimize? $\mathcal{L}^{(j)}(\vec{\theta}_t) = \frac{1}{N_{\mathcal{D}^{(j)}}} \sum_{i=1}^{N_{\mathcal{D}^{(j)}}} \mathcal{L}_i\left(\vec{y}_i^{(j)}, \bar{\vec{y}}_i^{(j)}\right)$
2. Neural network architecture
3. **Training algorithm**

Parameters Optimization with Gradient Descent

1. **Forward Pass**

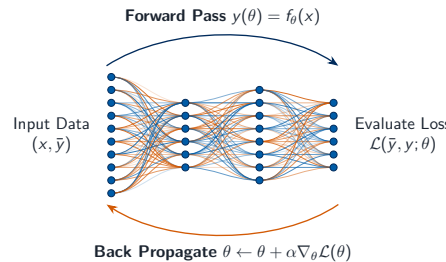
$$\bar{\vec{y}}_i^{(j)} = f_{\vec{\theta}_t}(\vec{x}_i^{(j)})$$

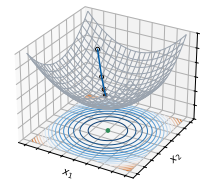
2. **Loss calculation**

$$\mathcal{L}^{(j)}(\vec{\theta}_t) = \frac{1}{N_{\mathcal{D}(j)}} \sum_{i=1}^{N_{\mathcal{D}(j)}} \mathcal{L}_i(\vec{y}_i^{(j)}, \bar{\vec{y}}_i^{(j)})$$

3. **Backpropagation**

$$\vec{\theta}_{t+1} \leftarrow \vec{\theta}_t - \eta \frac{\partial \mathcal{L}^{(j)}(\vec{\theta}_t)}{\partial \vec{\theta}_t}.$$





Mini-Batches Instead of the Full Dataset

The exact gradient needs *every* training example:

$$\nabla_{\vec{\theta}} \mathcal{L}(\vec{\theta}; \mathcal{D}) = \frac{1}{N} \sum_{i=1}^{N} \nabla_{\vec{\theta}} \ell(f_{\vec{\theta}}(\vec{x}_i), \vec{y}_i)$$

- **memory:** the forward pass stores the intermediate values of every example for backpropagation; real datasets do not fit in GPU memory (ImageNet: 1.28 million images, $>$ 100 GB)
- **cost:** one update would need a pass over the whole dataset — very few updates per hour

Mini-batch gradient

Average over a random subset of $B \ll N$ examples:

$$\nabla_{\vec{\theta}} \mathcal{L} \approx \frac{1}{B} \sum_{i \in \text{batch}} \nabla_{\vec{\theta}} \ell_i$$

a noisy but unbiased estimate — and cheap

Mini-Batches, Iterations, Epochs



One epoch:

1. shuffle the training data
2. split into batches of size B
3. for each batch: forward pass $\rightarrow$ loss on the batch $\rightarrow$ gradient $\rightarrow$ update $\vec{\theta}$ *(one iteration)*
4. stop when every example has been used once

Example: MNIST

$N = 50\,000$ training images, $B = 64$
$\Rightarrow \lceil 50\,000/64 \rceil = 782$ iterations per epoch
20 epochs $\Rightarrow$ 15 640 updates of $\vec{\theta}$

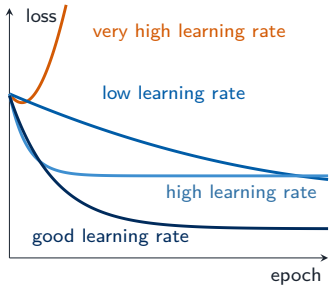
Batch size, learning rate and number of epochs are *hyperparameters*: we choose them, the training does not.

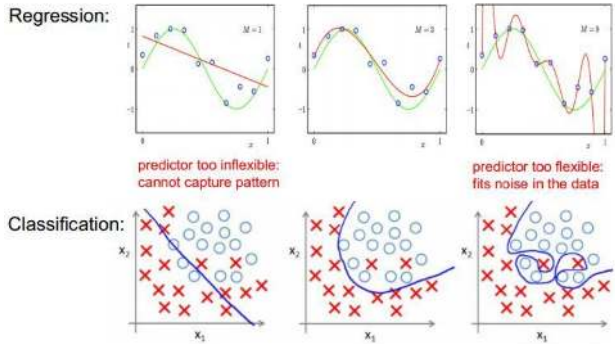
Minibatch Stochastic Gradient Descent (SGD)

Algorithm 1 Minibatch stochastic gradient descent (SGD)

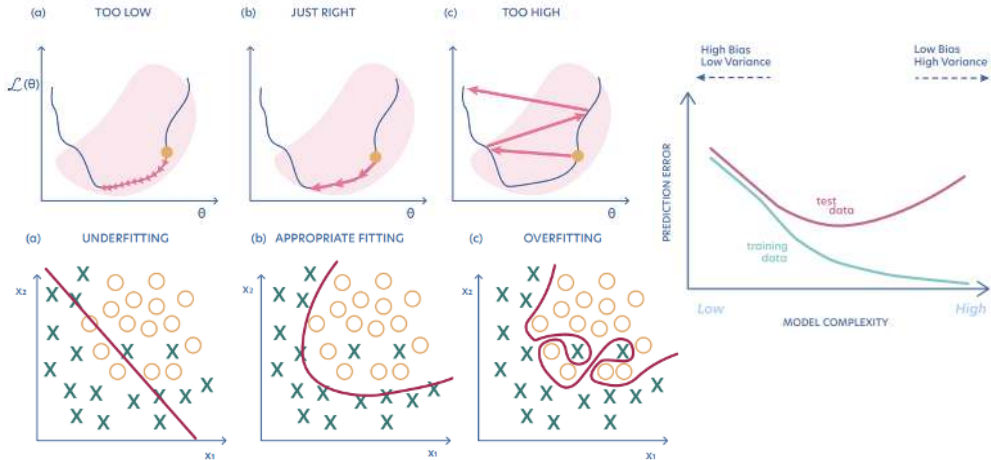
Require: Learning rate η
  Initialize $\boldsymbol{\theta}$ to random values
  for epoch $= 1$ to no_epochs **do**
    Shuffle $\mathcal{D}_{\text{train}}$
    for i $= 1$ to m (where m is a minibatch size) **do**
      $\boldsymbol{x}_i, y_i \sim \mathcal{D}_{\text{train}}$ ▷ Draw random data point from data set without replacement
      $\mathcal{L} \leftarrow \frac{1}{m} \sum_{i=1}^{m} \mathcal{L}(y_i, f(\boldsymbol{x}_i))$ ▷ Compute loss function on the minibatch
      $(\nabla \mathcal{L})_j \leftarrow \frac{\partial \mathcal{L}}{\partial \theta_j}$ ▷ Compute gradients
      $\theta_j \leftarrow \theta_j - \eta \frac{\partial}{\partial \theta_j} \mathcal{L}$ ▷ Update parameters
    end for
  end for
  return $\boldsymbol{\theta}$

A “Well-Trained” Neural Network





A "Well-Trained" Neural Network



2

The Success of Neural Networks



JAGIELLONIAN UNIVERSITY
IN KRAKOW

Neural Networks Learn Probability Distribution of Training Data

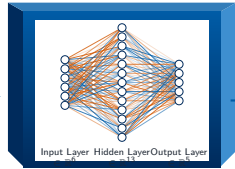
Data:
- labeled images
- text
- video
- tabular data

"Real world" data come from some "real"/"given" probability distribution



Neural Networks Learn Probability Distribution of Training Data

Data:
- labeled images
- text
- video
- tabular data



Probability distribution of:
- given label for classification task
- next word for text generation
- set of pixels for image generation
- next video frame for video generation (information about spatial and temporal structure)

"Real world" data come from some "real"/"given" probability distribution

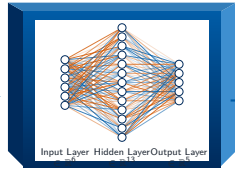


1. "Predicted" PD depends on trainable parameters
2. Compare learned PD with true PD – calculate loss function

NN parameters update

Neural Networks Learn Probability Distribution of Training Data

Data:
- labeled images
- text
- video
- tabular data



Probability distribution of:
- given label for classification task
- next word for text generation
- set of pixels for image generation
- next video frame for video generation (information about spatial and temporal structure)

1. "Predicted" PD depends on trainable parameters
2. Compare learned PD with true PD – calculate loss function

NN parameters update

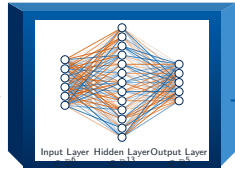
"Bob likes to eat . . . "

"Real world" data come from some "real"/"given" probability distribution



Neural Networks Learn Probability Distribution of Training Data

Data:
- labeled images
- text
- video
- tabular data



Probability distribution of:
- given label for classification task
- next word for text generation
- set of pixels for image generation
- next video frame for video generation (information about spatial and temporal structure)

1. "Predicted" PD depends on trainable parameters
2. Compare learned PD with true PD – calculate loss function

NN parameters update

"Real world" data come from some "real"/"given" probability distribution



"Bob likes to eat . . . "
"Bob likes to eat red . . . "

Today's "AI"



1. **Learning** a probability distribution from the provided data
2. **Sampling** from the learned probability distribution

Deep Learning Now: Enabling Factors

1. **Widespread availability of data and computational power**: Deep learning relies on access to vast amounts of high-quality training data. The proliferation of the Internet and mobile devices has made inexpensive, high-quality data widely available. For instance, in 2024 alone, Facebook generates 4,000 terabytes of data.

2. **Affordable computational power**: The advancements in graphics cards (GPUs) optimized for numerical computations, especially for fast single-precision matrix multiplications, have made training deep learning models more accessible.

3. **User-friendly programming languages**: Programming languages such as Python and Julia enable efficient prototype code development with a low learning threshold. This accessibility allows researchers and developers to quickly train simple models without needing to learn a complex API.

Deep Learning Now: Enabling Factors

4. **Automation of gradient computation**: Neural network training heavily depends on computing gradients for parameter optimization. Previously, gradient computation was done manually for each network architecture, which was time-consuming and error-prone. With the introduction of automatic differentiation algorithms in modern programming tools, developers can now prototype and test new architectures with greater ease and speed.

5. **Proper labeling of training data**: Properly labeled data is crucial for training deep learning models. This task is often performed by low-cost labor in developing countries. While this aspect is frequently overlooked, it plays a vital role in the success of deep learning. Contributions of these workers, despite their low pay, are essential to the development of AI systems.

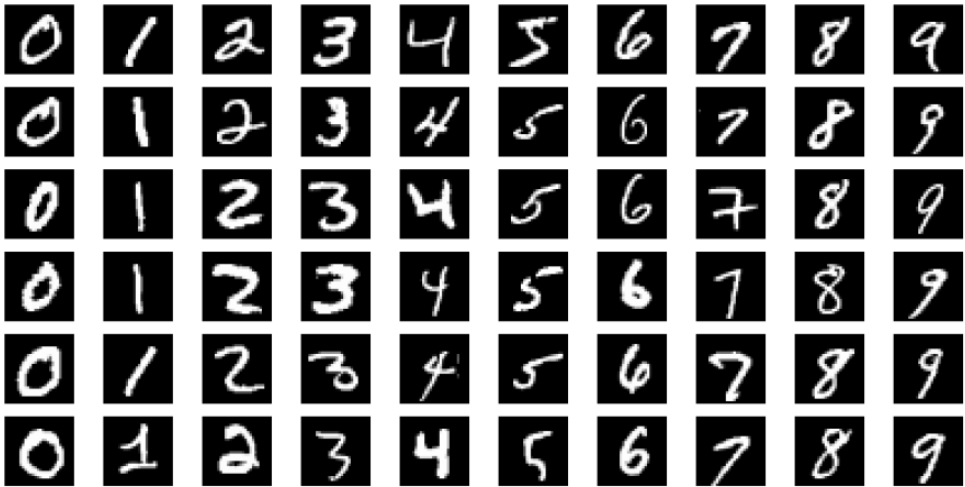


3

Example #1: *Hello World!* MNIST

JAGIELLONIAN UNIVERSITY
IN KRAKÓW

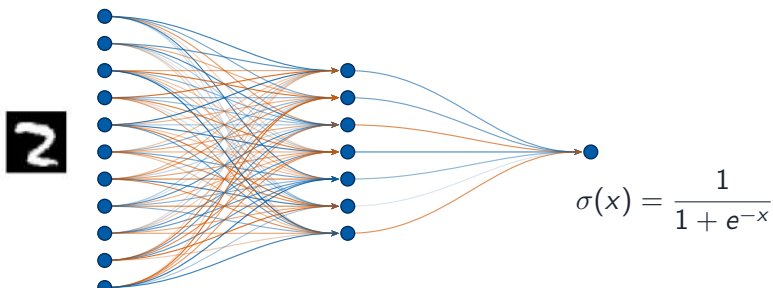
Example #1: *Hello world!* MNIST



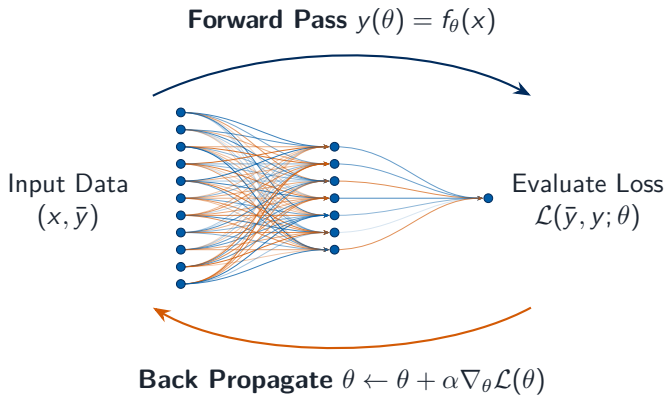
Digit Classification as Nonlinear Function Fitting

We consider a data set of n tuples (x_i, y_i) that we denote as $X \in \mathbb{R}^{n \times m}$ and $Y \in \mathbb{R}^n$. Here, m denotes the number of features in our samples. The perceptron consists on:

1. A linear transformation $f : \mathbb{R}^m \mapsto \mathbb{R}^h$ of the form $z_i = x_i^T W + \mathbf{b}$. Here, $W \in \mathbb{R}^{m \times h}$ are the weights, and $\mathbf{b} \in \mathbb{R}^h$ are the biases.
2. A nonlinear transformation $\hat{y}_i = \sigma(z_i)$, in this case: the sigmoid function $\sigma(z_i) = \frac{1}{1+e^{-z_i}}$.

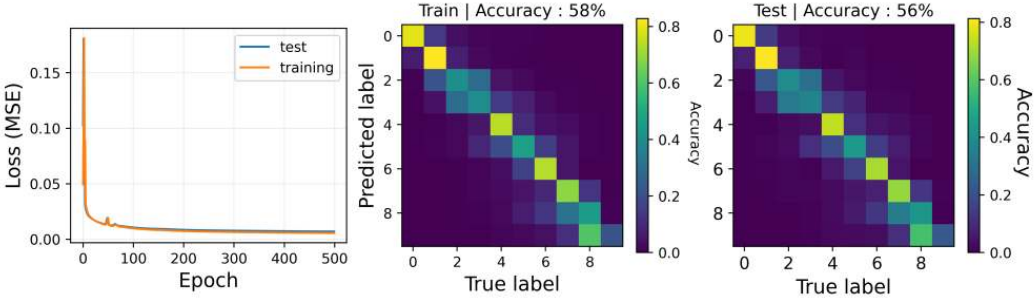


Parameters Update



Training Results

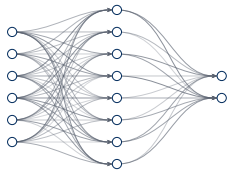
Good? Bad?



Loss Function: Categorical Cross-Entropy



p(x) = [0,0,1,0,0,0,0,0,0,0]



q("digit" | x)

q(x) = [0.1, 0, 0.7, 0.01, 0.1, 0.02, 0, 0.02, 0, 0.05]
// normalization

$p(x)$: Probability distribution of training data

$q(x)$: Probability distribution given by NN

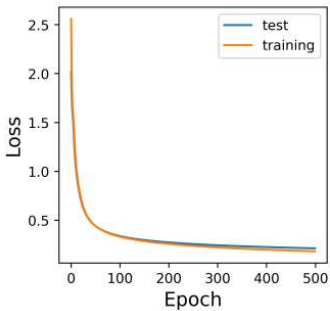
Kullback – Leibler divergence: $D_{\mathrm{KL}}(p\|q) = \left\langle \log \frac{p(x)}{q(x)} \right\rangle_p = \frac{1}{n} \sum_i p(x_i) \log \frac{p(x_i)}{q(x_i)}$

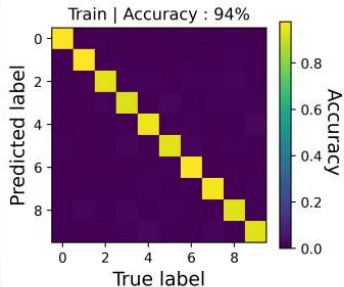
1. Vanishes for identical $q(x) = p(x)$

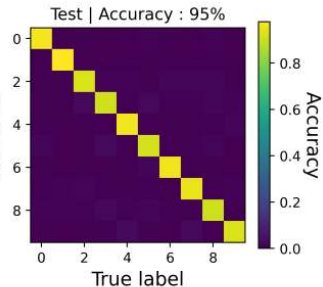
2. Asymmetric $D_{\mathrm{KL}}(p\|q) \neq D_{\mathrm{KL}}(q\|p)$

$$D_{\mathrm{KL}}(p\|q) = \frac{1}{n} \sum_i p(x_i) \log p(x_i) - \frac{1}{n} \sum_i p(x_i) \log q(x_i) =: \cancel{S(p)} + \underbrace{\overbrace{\mathcal{L}_{\mathrm{CE}}(p,q)}^{\text{Categorical cross-entropy}}}_{\text{Loss function of interest}}$$

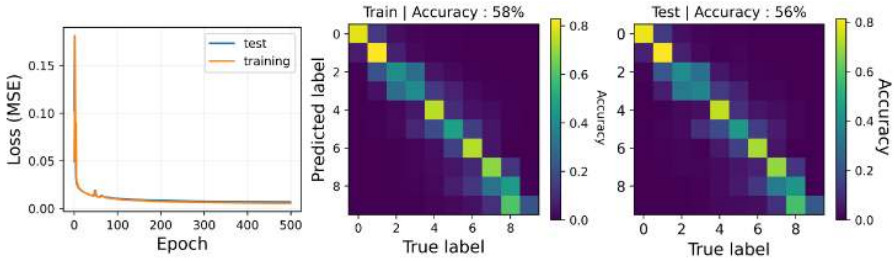
Loss Function: Categorical Cross-Entropy







MNIST


test
training
Loss (MSE)
Epoch
Train | Accuracy : 58%
Predicted label
True label
Accuracy
Test | Accuracy : 56%
True label
Accuracy

Summary

1. **Loss functions**: mean squared error for regression, cross-entropy for classification
2. **Training is optimisation**: gradient descent on the parameters, with mini-batches, iterations and epochs (stochastic gradient descent)
3. **A well-trained network generalises**: low loss on data it has not seen
4. **Neural networks learn a probability distribution** from a finite set of samples of an unknown distribution
5. **MNIST from scratch**: digit classification as non-linear function fitting

Next Lecture

1. Deep learning "Hello world" #2: CIFAR-10
2. Improving neural-network predictions: regularisation techniques

Please read Chapters 1 and 2 of the Lecture Notes

Thank You!

Questions, project ideas, feedback: marcin.plodzien@uj.edu.pl

Website · GitHub · Google Scholar

